Ders 18

ÖRNEK PYTHON KURSU

List Comprehension

Bölüm 4 · Veri Yapıları

Kurs	*Bölüm*	*Tarih*
Örnek Python Kursu	Bölüm 4 - Veri Yapıları	8 Kasım 2026

Ders transkriptinden derlenmiştir · Kişisel kullanım içindir

İçindekiler

Bu notta yer alan başlıklar

List Comprehension 3

- Özet 3
- Klasik Döngü ile Karşılaştırma 3
- Koşul (Filtre) Eklemek 3
- if-else Kullanımı 3
- Küme ve Sözlük Üreteçleri 4
- Önemli Kavramlar 4
- Dikkat Edilmesi Gerekenler 4
- Tekrar Soruları 4

List Comprehension

List comprehension (liste üreteci), var olan bir yinelenebilir (iterable) nesneden tek satırda yeni bir liste oluşturmanın kısa yoludur. Bu derste temel söz dizimi, filtre koşulu ekleme, `if-else` kullanımındaki yaygın hata ve aynı yapının küme ile sözlüklere uygulanması anlatılıyor.

1. Klasik Döngü ile Karşılaştırma

1'den 5'e kadar sayıların karelerini almak istediğimizi düşünelim. Klasik yöntemde boş bir liste oluşturulur, döngüyle her eleman gezilir ve sonuç `append` ile eklenir:

```
numbers = [1, 2, 3, 4, 5]
squares = []
for x in numbers:
    squares.append(x * x)
```

Aynı iş list comprehension ile tek satırda yapılır. Köşeli parantez içine önce **ifade**, ardından **for** kısmı yazılır:

```
squares = [x * x for x in numbers]   # [1, 4, 9, 16, 25]
```

2. Koşul (Filtre) Eklemek

Yalnızca belirli elemanları tutmak için sona bir `if` koşulu eklenir. Örneğin sadece çift sayılar:

```
evens = [x for x in numbers if x % 2 == 0]   # [2, 4]
```

3. if-else Kullanımı

Bir `else` dalı gerekiyorsa koşullu ifade sona değil, **`for`'dan önce, en başa** yazılır:

```
labels = ["çift" if x % 2 == 0 else "tek" for x in numbers]
```

Amaç	Koşulun yeri	Örnek
Eleman filtrelemek	Sonda	`[x for x in nums if x > 0]`
Değeri dönüştürmek (if-else)	Başta	`[x if x > 0 else 0 for x in nums]`

4. *Küme ve Sözlük Üreteçleri*

Aynı söz dizimi diğer veri yapılarında da çalışır:

```
unique_squares = {x * x for x in numbers}       # küme: süslü parantez
square_map = {x: x * x for x in numbers}        # sözlük: anahtar: değer
```

Önemli Kavramlar

Kavram	Açıklama
Yinelenebilir (iterable)	Üzerinde döngü kurulabilen nesne: liste, demet, string, range vb.
İfade (expression)	Her eleman için hesaplanıp yeni listeye eklenen değer (`x * x`)
Filtre koşulu	Sondaki `if`; hangi elemanların dahil edileceğini belirler

Dikkat Edilmesi Gerekenler

> **DİKKAT**
> `if-else` yapısını sona yazmak söz dizimi hatasına yol açar. `else` varsa koşul en başa gelir.

> **ABARTMAYIN**
> iç içe üç döngü ve birden fazla koşul içeren bir comprehension okunmaz hale gelir. Bu durumda normal `for` döngüsü kullanın; okunabilirlik önceliklidir.

Tekrar Soruları

i. List comprehension'ın klasik `for` + `append` yöntemine göre avantajı nedir?

ii. Sadece pozitif sayıları içeren bir liste üretmek için koşul nereye yazılır?

iii. `else` dalı gerektiğinde koşullu ifade neden `for`'dan önce gelir?

iv. Bir sözlük üreteci nasıl yazılır?

v. Hangi durumda comprehension yerine normal döngü tercih edilmelidir?